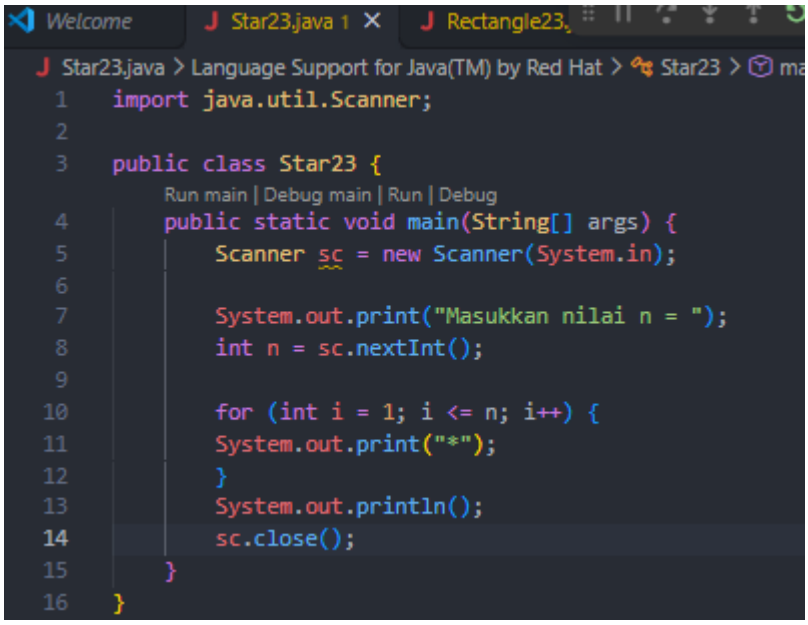
JOBSHEET 8

Nama	Muhammad Panji Taqiyuddin
NIM	264107020024
Kelas	Teknik Informatika 1C
Absen	23

Percobaan 1: Review Perulangan 1




```
Masukkan nilai n = 5
*****
```

Pertanyaan

1. Jika inisialisasi i = 1 diubah menjadi i = 0 tanpa mengubah kondisi, berapa karakter * yang tercetak ketika n = 5? Jelaskan.

   Jawab : Bakal tercetak 6 buah bintang (*). Karena i mulainya dari angka 0, terus jalannya sampai angka 5 $(0,1,2,3,4,5)$. Kalau dihitung, totalnya ada 6 kali putaran, jadi otomatis bintang yang keluar ada 6 biji.
2. Jika kondisi i <= n diganti menjadi i < n, berapa kali perulangan dijalankan saat n = 5? Mengapa hasilnya berbeda?

   Jawab : Perulangannya cuma jalan 4 kali (buat nilai i = 1, 2, 3,4).

   Bedanya gara-gara tanda kurang dari (<) itu nggak ngikutin angkanya. Kalau pakai (<=) (kecil sama dengan), angka 5-nya masih ikut kehitung. Tapi kalau

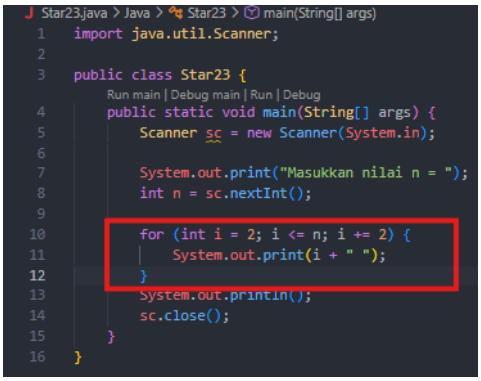
cuma pakai <, pas i nyampe angka 5, syaratnya langsung dianggap salah (false) alias disuruh berhenti sama sistem sebelum angka 5-nya diproses. Makanya jumlah putarannya berkurang jadi 4 kali.

3. Prediksi output potongan kode berikut ketika n = 9, kemudian buktikan dengan mengganti perulangan FOR pada program Anda dan menjalankannya.

```
for (int i = 2; i <= n; i += 2) {
    System.out.print(i + " ");
}
```

Modifikasi kode tersebut agar mencetak bilangan ganjil positif yang tidak melebihi n.

Jawab : Programnya disuruh mulai nge-print dari angka 2, terus tiap mutar dia nambah 2 angka (i += 2) selama i masih kurang dari atau sama dengan 9. Jadi angka yang lolos seleksi cuma si genap: 2, 4, 6, dan 8. Begitu mau masuk angka 10, kan udah lewat dari 9, jadi otomatis berhenti deh.




```
Masukkan nilai n = 9
2 4 6 8
```

4. Jelaskan mengapa perubahan increment i++ menjadi i-- (dengan kondisi tetap i <= n) membuat perulangan berjalan terus-menerus dan tidak berhenti dalam waktu wajar untuk n positif. Jangan menjalankan versi tersebut; jika terlanjur berjalan, hentikan program dengan menekan Ctrl + C pada terminal.

Jawab : Bayangin, kita punya angka awal 1 dan syarat berhentinya kalau i <= 5. Harusnya kan kalau mau berhenti, angkanya nambah jadi 2, 3, 4, 5 (mendekati target).

Nah, kalau malah dikasih i--, angkanya malah terus minus/berkurang (1, 0, -1, -2, -3, dan seterusnya). Otomatis angka minus itu bakal selamanya lebih kecil dari 5. Karena syaratnya nggak bakal pernah salah, programnya jadi mutar-mutar terus nggak ada ujungnya. Makanya bakal macet.

Percobaan 2: Pola Bintang Persegi Panjang (Nested FOR)

Rectangle23.java > Java > Rectangle23 > main(String[] args)

```java
import java.util.Scanner;

    public class Rectangle23 {
        Run main | Debug main | Run | Debug
        public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan jumlah baris: ");
        int baris = sc.nextInt();
        System.out.print("Masukkan jumlah kolom: ");
        int kolom = sc.nextInt();

        for (int i = 1; i <= baris; i++) {
            for (int j = 1; j <= kolom; j++) {
            System.out.print("*");
            }
        System.out.println();
        }
    sc.close();
    }
}
```

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
******
******
******
```

Pernyataan

1. Tentukan fungsi variabel i dan j pada program tersebut.

   Jawab :

   - **Variabel i** fungsinya buat ngatur baris (perulangan luar/outer loop).
   - **Variabel j** fungsinya buat ngatur kolom atau jumlah bintang ke samping di setiap barisnya (perulangan dalam/inner loop).

2. Berapa kali outer loop, inner loop, dan perintah print("*") dijalankan untuk baris = 4 dan kolom = 7?

   Jawab :

   - **Outer loop**: Dijalankan sebanyak 4 kali (sesuai jumlah baris).
   - **Inner loop**: Dijalankan sebanyak 28 kali totalnya (karena 4 baris dikali 7 kolom). Maksudnya, si inner loop bakal ngereset dan jalan 7 kali di setiap 1 kali putaran outer loop.

- **Perintah print("*"):** Dijalankan sebanyak 28 kali juga (karena dipanggil sekali di dalam inner loop).

3. Apa yang terjadi jika batas perulangan luar dan dalam ditukar (outer menggunakan kolom, inner menggunakan baris)? Gunakan baris = 3 dan kolom = 6 untuk menjelaskan.

   Jawab :

   Kalau dibalik jadi outer pakai kolom (6) dan inner pakai baris (3), bentuk pola bintangnya bakal berbalik arah.

   - Yang tadinya bikin persegi panjang 3 baris ke bawah dan 6 kolom ke samping, bakal berubah jadi 6 baris ke bawah dan 3 kolom ke samping.
   - Kenapa? Karena perulangan luarnya sekarang mutar 6 kali (bikin 6 baris), dan di tiap barisnya dicetak 3 bintang ke samping.

```
for (int i = 1; i <= kolom; i++) { //ini outer loop baris
    for (int j = 1; j <= baris; j++) { //ini outer loop kolom
    System.out.print("*");
    }
```

```
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
***
***
***
***
***
***
```

```
Masukkan jumlah baris: 6
Masukkan jumlah kolom: 3
******
******
******
```

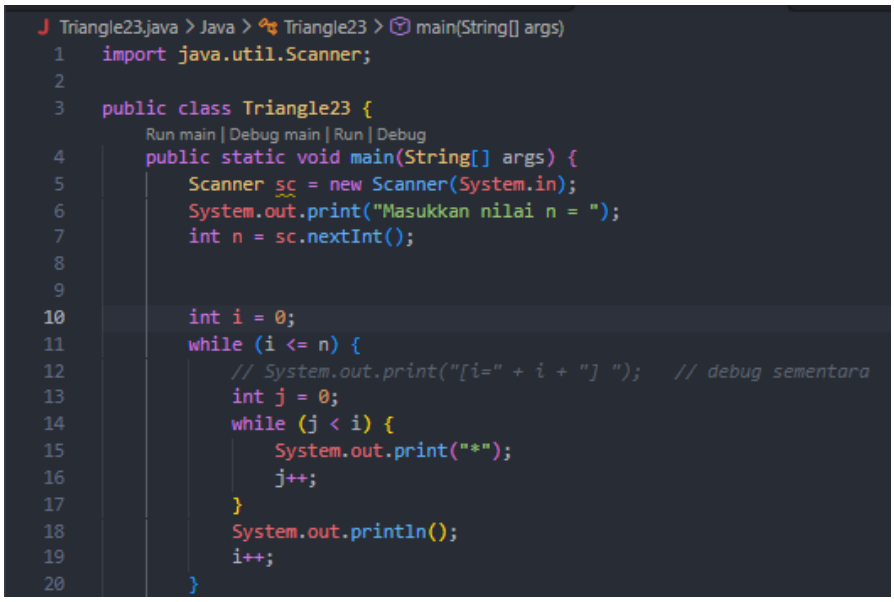
4. Jelaskan akibat penghapusan System.out.println(); setelah inner loop.

   Jawab : Akibatnya, semua bintang bakal jejer ke samping jadi satu baris panjang tanpa ada enter/turun baris ke bawah.

```
Masukkan jumlah baris: 6
Masukkan jumlah kolom: 3
******************
```

5. Modifikasi program agar karakter yang dicetak adalah #, lalu commit dan push perubahan ke GitHub.

Percobaan 3: Debugging Bintang Segitiga – Nested WHILE



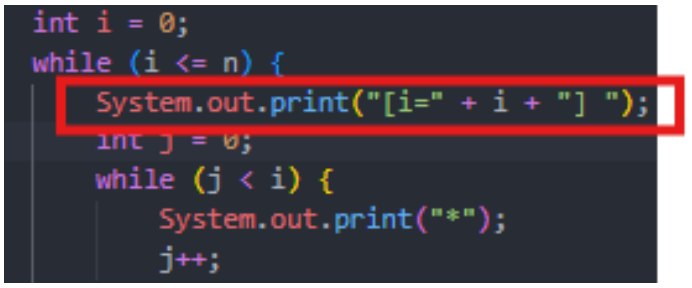
```java
import java.util.Scanner;

public class Triangle23 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();


        int i = 0;
        while (i <= n) {
            // System.out.print("[i=" + i + "] ");   // debug sementara
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i++;
        }
```

```
Masukkan nilai n = 5
*
**
***
****
*****
```

Untuk mencari penyebab kesalahan ada di bawah ini



```java
int i = 0;
while (i <= n) {
    System.out.print("[i=" + i + "] ");
    int j = 0;
    while (j < i) {
        System.out.print("*");
        j++;
```

```
Masukkan nilai n = 5
[i=0]
[i=1] *
[i=2] **
[i=3] ***
[i=4] ****
[i=5] *****
```

1. Jalankan kembali program dengan n = 5,
   lalu amati hasilnya:
   a. Berapa kali outer loop dijalankan? Apakah sama dengan jumlah baris yang diinginkan?
      Jawab : Outer loop dijalankan sebanyak 6 kali (dari $i = 0$ sampai $i = 5$). Apakah sama? Nggak sama, karena kita mintanya $n = 5$ baris, tapi programnya malah ngehasilin 6 baris gara-gara mulainya dari angka 0 $(0,1,2,3,4,5)$.
   b. berapa karakter * yang tercetak saat i = 0?
      Jawab : Tidak ada (0 karakter *).
   c. mengapa semua karakter tampil dalam satu baris?
      Jawab; Karena di dalam perulangan dalamnya kita pakai System.out.print("*") yang sifatnya nyetak ke samping terus tanpa bikin enter baru, kecuali kalau ditambahin perintah enter (println) di luar perulangan dalam.

2. Perbaiki kode satu per satu dan jalankan program setelah setiap perbaikan:
   a. tambahkan System.out.println(); setelah inner loop selesai, sebelum i++. Jalankan dan amati: apakah masih ada baris yang tidak berisi *?
      Jawab : Setelah ditambahin, garisnya udah bener pada turun ke bawah. Apakah masih ada baris yang tidak berisi ? Masih, yaitu pas di baris paling atas waktu $i = 0$ (cuma keluar tulisan [i=0] doang tanpa bintang).
   b. ubah inisialisasi dan/atau kondisi outer loop agar perulangan berjalan tepat n kali tanpa baris kosong. Jalankan kembali.
      Jawab :

```java
import java.util.Scanner;

public class Triangle23 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();


        int i = 1;
        while (i <= n) {
            System.out.print("[i=" + i + "] ");
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i++;
        }
```

```
Masukkan nilai n = 5
[i=1] *
[i=2] **
[i=3] ***
[i=4] ****
[i=5] *****
```

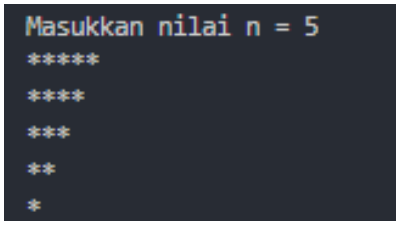
Jalankan program dengan n = 5. Bagian segitiga terbalik harus serupa dengan tampilan berikut.

```java
import java.util.Scanner;

public class Triangle23 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        int i = 0;
```

```java
        i = n;
        while (i >= 1) {
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i--;
        }
        sc.close();
    }
}
```



Pertanyaan

1. Sebutkan ketidaksesuaian output awal dan minimal dua perbaikan yang diperlukan untuk menghasilkan pola segitiga bertingkat.

   Jawab : Pada kode awal, terdapat dua ketidaksesuaian utama:

   Tidak terdapat System.out.println() setelah inner loop. Akibatnya, bintang dari setiap iterasi outer loop dicetak secara berurutan pada satu baris, sehingga pola segitiga tidak terbentuk.

   Kondisi inner loop j < i membentuk segitiga normal, bukan segitiga terbalik. Jumlah bintang bertambah pada setiap baris, sedangkan segitiga terbalik membutuhkan jumlah bintang yang berkurang.

   Perbaikan yang diperlukan adalah:

   - Menambahkan System.out.println() setelah inner loop agar setiap baris dicetak pada baris baru.
   - Mengubah kondisi inner loop menjadi j < n - i agar jumlah bintang berkurang pada setiap baris.

2. Jelaskan peran variabel i dan j. Mengapa nilai j harus diinisialisasi kembali ke 0 pada setiap iterasi outer loop? Apa yang terjadi jika j tidak di-reset?

   Jawab ; Variabel i berfungsi sebagai pencacah outer loop atau perulangan luar yang menentukan baris yang sedang dicetak.

   - Variabel j berfungsi sebagai pencacah inner loop atau perulangan dalam yang menentukan jumlah bintang pada setiap baris.
   - Variabel j harus diinisialisasi kembali menjadi 0 pada setiap iterasi outer loop agar perhitungan jumlah bintang dimulai dari awal untuk setiap baris.

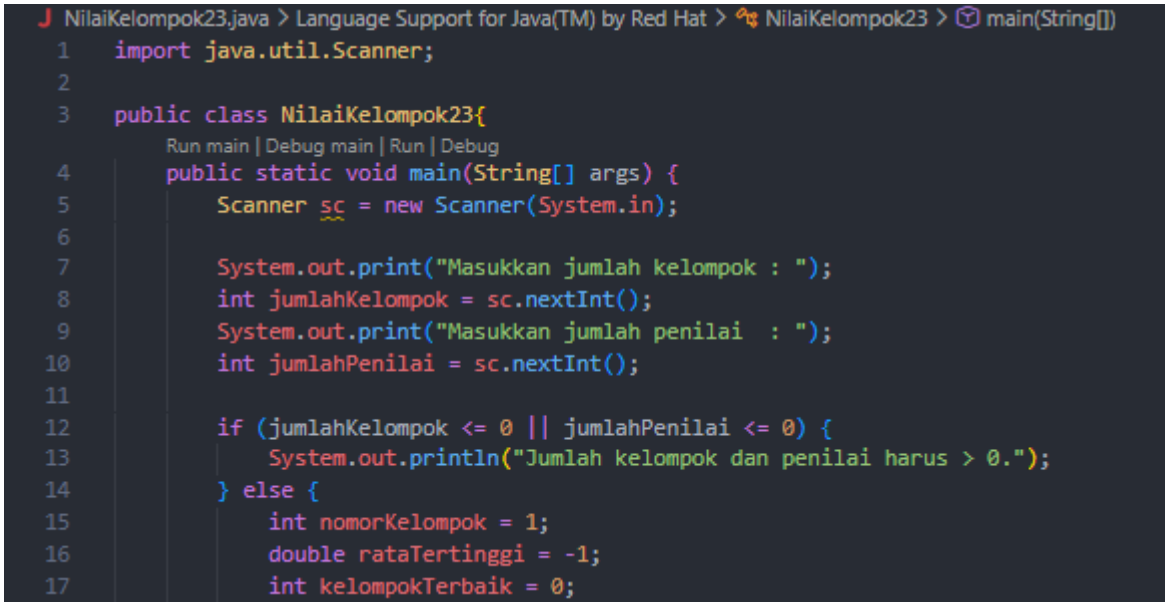
   Jika j tidak di-reset, nilai j akan tetap membawa hasil iterasi sebelumnya. Akibatnya, kondisi inner loop mungkin langsung bernilai salah dan bintang tidak tercetak pada baris berikutnya, sehingga pola segitiga tidak terbentuk dengan benar.

3. Jelaskan perubahan inisialisasi, kondisi, dan perubahan nilai (update) pada outer loop yang diperlukan untuk membentuk segitiga terbalik.

   Jawab ;

   - Inisialisasi i = 0: perulangan dimulai dari indeks baris pertama.
   - Kondisi i <= n: perulangan berjalan selama i kurang dari atau sama dengan n. Kondisi ini menghasilkan n + 1 iterasi, termasuk satu baris kosong terakhir.
   - Update i++: nilai i bertambah satu pada setiap iterasi, sehingga jumlah bintang yang dicetak oleh inner loop berkurang satu karena batasnya adalah n - i.

Percobaan 4: Studi Kasus Nilai Tugas Proyek Kelompok



```
            while (nomorKelompok <= jumlahKelompok) {
                System.out.println("Kelompok " + nomorKelompok);
                double totalNilai = 0;

                for (int j = 1; j <= jumlahPenilai; j++) {
                    int nilai;
                    do {
                        System.out.print("  Nilai dari Penilai " + j + ": ");
                        nilai = sc.nextInt();
                        if (nilai < 0 || nilai > 100) {
                            System.out.println("  Nilai harus 0-100. Ulangi input!");
                        }
                    } while (nilai < 0 || nilai > 100);
                    totalNilai += nilai;
                }

                double rataNilai = totalNilai / jumlahPenilai;
                System.out.println("Kelompok " + nomorKelompok
                        + ": nilai rata-rata = " + rataNilai);

                if (rataNilai > rataTertinggi) {
                    rataTertinggi = rataNilai;
                    kelompokTerbaik = nomorKelompok;
                }

                nomorKelompok++;
            }

            System.out.println("============================");
            System.out.println("Kelompok terbaik    : " + kelompokTerbaik);
            System.out.println("Rata-rata tertinggi : " + rataTertinggi);
        }
        sc.close();
    }
}
```

Output

```
Masukkan jumlah kelompok : 3
Masukkan jumlah penilai  : 2
Kelompok 1
  Nilai dari Penilai 1: 80
  Nilai dari Penilai 2: 90
Kelompok 1: nilai rata-rata = 85.0
Kelompok 2
  Nilai dari Penilai 1: 85
  Nilai dari Penilai 2: 100
Kelompok 2: nilai rata-rata = 92.5
Kelompok 3
  Nilai dari Penilai 1: 90
  Nilai dari Penilai 2: 101
  Nilai harus 0-100. Ulangi input!
  Nilai dari Penilai 2: 96
Kelompok 3: nilai rata-rata = 93.0
============================
Kelompok terbaik    : 3
Rata-rata tertinggi : 93.0
```

Pertanyaan

1. Mengapa totalNilai perlu di-reset di dalam outer loop, tetapi rataTertinggi tidak boleh di-reset pada setiap kelompok?

   Jawab : totalNilai perlu di-reset menjadi 0 pada setiap awal kelompok karena variabel ini digunakan untuk menjumlahkan nilai dari para penilai untuk satu kelompok saja. Jika tidak di-reset, nilai kelompok sebelumnya akan ikut terakumulasi ke kelompok berikutnya sehingga rata-rata menjadi salah.

   rataTertinggi tidak boleh di-reset pada setiap kelompok karena variabel ini digunakan untuk menyimpan rata-rata tertinggi dari seluruh kelompok yang sudah diproses. Nilainya harus dipertahankan agar dapat dibandingkan dengan rata-rata kelompok berikutnya.
2. Untuk 3 kelompok dan 2 penilai (tanpa input yang salah), berapa kali badan perulangan FOR dijalankan secara keseluruhan?

   Jawab ; Setiap kelompok menjalankan perulangan FOR sebanyak 2 kali, satu kali untuk setiap penilai. Karena terdapat 3 kelompok, totalnya adalah 6 kali, dengan asumsi tidak ada input yang salah yang menyebabkan pengulangan tambahan pada do-while.
3. Apa yang terjadi jika totalNilai dideklarasikan sebagai int dan rata-rata dihitung menggunakan pembagian dua bilangan bulat? Berikan contoh nilai.

   Jawab : ika rata-rata dihitung menggunakan pembagian bilangan bulat, bagian desimal akan dihilangkan sehingga hasil rata-rata menjadi kurang akurat.

   Misal: 161 / 2 = 80

   Hasil yang seharusnya adalah 80.5, tetapi karena kedua operand bertipe int, pembagian menghasilkan 80 terlebih dahulu sebelum disimpan ke variabel double.
4. Mengapa rataTertinggi diberi nilai awal -1, bukan 0? Berikan contoh input yang membuat hasil salah apabila nilai awalnya 0.

   Jawab : rataTertinggi diberi nilai awal -1 agar rata-rata kelompok pertama pasti lebih besar daripada nilai awal tersebut, sehingga kelompok pertama selalu tercatat sebagai kelompok terbaik sementara.

Jika nilai awalnya 0, program dapat menghasilkan kelompok terbaik yang salah apabila semua kelompok memiliki rata-rata 0 dan pembaruan menggunakan operator >.

5. Program ini memiliki tiga tingkat perulangan. Sebutkan masing-masing jenis perulangan beserta perannya.

   Jawab : Tingkat Jenis perulangan peran

   Luar While Memproses setiap kelompok sampai seluruh kelompok selesai dinilai.

   Tengah For Meminta dan menjumlahkan nilai dari setiap penilai untuk kelompok yang sedang diproses.

   Dalam Do-while Meminta input nilai kembali jika nilai yang dimasukkan kurang dari 0 atau lebih dari 100.

TUGAS

1. Buat program untuk mencetak pola bilangan 1 sampai nomor baris. Masukkan bilangan bulat n (n minimal 1). Pada baris pertama tercetak 1, pada baris kedua 1 2, dan seterusnya sampai baris ke-n. Setelah pola selesai, tampilkan total banyaknya angka yang telah dicetak. Simpan dengan nama file: Tugas1NoAbsen.java. Contoh untuk n = 5:

```
Masukkan nilai n = 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Total angka tercetak: 15
```

Kode Program

Tugas1_23.java > Language Support for Java(TM) by Red Hat > Tugas1_23 > main(String[])

```java
import java.util.Scanner;

public class Tugas1_23{
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        if (n < 1) {
            System.out.println("n minimal 1.");
        } else {
            int total = 0;
            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= i; j++) {
                    if (j > 1) {
                        System.out.print(" ");
                    }
                    System.out.print(j);
                    total++;
                }
                System.out.println();
            }
            System.out.println("Total angka tercetak: " + total);
        }
        sc.close();
    }
}
```

```
Masukkan nilai n = 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Total angka tercetak: 15
```

2. Tugas 2: Tabel Perkalian

   Buat program untuk menampilkan tabel perkalian berukuran n × n menggunakan nested FOR atau WHILE. Nilai n minimal 3. Setiap posisi menampilkan hasil perkalian nomor baris dan nomor kolom. Atur spasi agar keluaran rapi, lalu tampilkan jumlah seluruh angka pada tabel. Simpan program dengan nama file: Tugas2NoAbsen.java.

   Contoh untuk n = 4:

```
Masukkan nilai n = 4
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
Jumlah seluruh angka: 100
```

Kode Program

Tugas2_23.java > Language Support for Java(TM) by Red Hat > Tugas2_23 > main(String[])

```java
import java.util.Scanner;

public class Tugas2_23 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan nilai n = ");
        int n = sc.nextInt();

        if (n < 3) {
            System.out.println("n minimal 3.");
        } else {
            int jumlah = 0;
            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= n; j++) {
                    int hasil = i * j;
                    System.out.printf("%4d", hasil);
                    jumlah += hasil;
                }
                System.out.println();
            }
            System.out.println("Jumlah seluruh angka: " + jumlah);
        }
        sc.close();
    }
}
```

```
Masukkan nilai n = 4
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
Jumlah seluruh angka: 100
```

3. Tugas 3: Rekap Penjualan Beberapa Toko

   Sebuah pusat perbelanjaan memiliki beberapa toko. Setiap toko melayani sejumlah pelanggan pada hari yang sama, dan setiap pelanggan membeli beberapa barang. Program meminta jumlah toko, kemudian jumlah pelanggan pada masing-masing toko, dan jumlah barang yang dibeli setiap pelanggan. Hitung dan tampilkan total pelanggan serta total barang per toko, total barang seluruh toko, dan nomor toko dengan jumlah barang terbanyak. Jika terdapat nilai tertinggi yang sama, tampilkan toko dengan nomor paling kecil. Simpan program dengan nama file: Tugas3NoAbsen.java.

   Ketentuan: jumlah toko minimal 1; jumlah pelanggan dan jumlah barang setiap pelanggan boleh 0, tetapi tidak boleh negatif. Gunakan perulangan bersarang; data tidak perlu disimpan satu per satu, cukup diakumulasikan ke variabel total.

   Contoh: Toko 1 memiliki 2 pelanggan yang membeli 3 dan 2 barang; Toko 2 memiliki 3 pelanggan yang membeli 1, 4, dan 2 barang. Ringkasan yang diharapkan:

```
Toko 1 - pelanggan: 2, total barang: 5
Toko 2 - pelanggan: 3, total barang: 7
Total barang seluruh toko: 12
Toko dengan barang terbanyak: 2
```

   Kode Program

Tugas3_23.java > Language Support for Java(TM) by Red Hat > Tugas3_23 > main(String[])

```java
import java.util.Scanner;

public class Tugas3_23 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Masukkan jumlah toko: ");
        int jumlahToko = sc.nextInt();

        if (jumlahToko < 1) {
            System.out.println("Jumlah toko minimal 1.");
        } else {
            int totalSemua = 0;
            int barangTertinggi = -1;
            int tokoTerbanyak = 0;
            String[] ringkasan = new String[jumlahToko];

            for (int t = 1; t <= jumlahToko; t++) {
                int jumlahPelanggan;
                do {
                    System.out.print("Toko " + t + " - jumlah pelanggan: ");
                    jumlahPelanggan = sc.nextInt();
                    if (jumlahPelanggan < 0) {
                        System.out.println("Tidak boleh negatif. Ulangi input!");
                    }
                } while (jumlahPelanggan < 0);

                int totalBarangToko = 0;
                for (int p = 1; p <= jumlahPelanggan; p++) {
                    int barang;
                    do {
                        System.out.print("  Pelanggan " + p + " - jumlah barang: ");
                        barang = sc.nextInt();
                        if (barang < 0) {
                            System.out.println("  Tidak boleh negatif. Ulangi input!");
                        }
                    } while (barang < 0);
                    totalBarangToko += barang;
                }

                ringkasan[t - 1] = "Toko " + t + " - pelanggan: " + jumlahPelanggan
                        + ", total barang: " + totalBarangToko;
                totalSemua += totalBarangToko;

                if (totalBarangToko > barangTertinggi) {
                    barangTertinggi = totalBarangToko;
                    tokoTerbanyak = t;
                }
            }

            System.out.println();
            for (int t = 0; t < jumlahToko; t++) {
                System.out.println(ringkasan[t]);
            }
            System.out.println("Total barang seluruh toko: " + totalSemua);
            System.out.println("Toko dengan barang terbanyak: " + tokoTerbanyak);
        }
        sc.close();
    }
}
```

```
Masukkan jumlah toko: 2
Toko 1 - jumlah pelanggan: 2
  Pelanggan 1 - jumlah barang: 3
  Pelanggan 2 - jumlah barang: 2
Toko 2 - jumlah pelanggan: 3
  Pelanggan 1 - jumlah barang: 6
  Pelanggan 2 - jumlah barang: 6
  Pelanggan 3 - jumlah barang: 0

Toko 1 - pelanggan: 2, total barang: 5
Toko 2 - pelanggan: 3, total barang: 12
Total barang seluruh toko: 17
Toko dengan barang terbanyak: 2
```